VAUTECH SOLUTIONS — AI INTERNSHIP REPORT

Task 3: Model Training with Controlled Experiments

Project Title:
Model Training with Controlled Experiments for Performance Optimization Using Scikit-learn

Intern Name:
Anurag Rathore

Intern ID:
VT26ML003

Department:
Artificial Intelligence & Data Science

Mentor:
Vishal Ramkumar Rajbhar

Company:
Vautech Solutions IT Solutions

Topic Name

Model Training with Controlled Experiments

Topic Description

This project focuses on training a supervised machine learning model using controlled experiments to analyze its performance. The main objective is to observe how changes in model parameters and feature selection affect accuracy. By systematically modifying one factor at a time, the project demonstrates how model behavior can be studied and optimized. Using Python and Scikit-learn, different experiments are conducted, compared, and documented to understand performance improvement in a structured and scientific way.

Abstract

This project presents an experimental approach to training supervised machine learning models using Python and the Scikit-learn library. The primary aim is to evaluate how different parameters and feature sets influence model performance. Controlled experiments are conducted by systematically modifying hyperparameters and selecting relevant features to observe their impact on accuracy. Multiple models are trained and tested on a dataset, and their results are compared using standard evaluation metrics. This structured methodology helps identify the most effective model configuration while promoting a deeper understanding of performance optimization techniques. The project highlights the importance of experimentation, analysis, and proper documentation in developing efficient and reliable machine learning systems

Introduction

Machine learning is a rapidly growing field of technology that enables computers to learn from data and make predictions with minimal human intervention. Among its various types, supervised learning is commonly used to solve real-world problems such as classification and prediction. Building an effective machine learning model requires careful training, evaluation, and continuous improvement to achieve reliable results.

This project focuses on model training with controlled experiments, a structured approach used to analyse how different parameters and feature selections impact model performance. By changing one factor at a time and comparing the outcomes, the project highlights the importance of systematic experimentation in improving accuracy and efficiency. Using Python and Scikit-learn, the study promotes a scientific and organized method for developing optimized machine learning models.

Problem Statement

Developing an accurate and reliable machine learning model is a challenging task because model performance is influenced by multiple factors such as parameter settings, feature selection, and training methods. Without a structured approach, it becomes difficult to determine which changes actually improve the model and which do not.

The problem addressed in this project is to systematically train and evaluate a supervised machine learning model using controlled experiments. By modifying one parameter at a time and comparing the results, the project aims to identify the most effective model configuration while ensuring that performance improvements are measurable, consistent, and data-driven.

Objectives

- To train a supervised machine learning model using Python and the Scikit-learn library.
- To analyse the impact of hyperparameter tuning on model performance.
- To compare different feature sets and identify the most relevant features for improving accuracy.
- To evaluate the model using standard performance metrics such as accuracy.
- To conduct controlled experiments by modifying one parameter at a time for systematic analysis.
- To document observations and results for better understanding of model optimization.

Dataset Description

The dataset used in this project is the Adult Income dataset, which is commonly applied in machine learning classification problems to predict whether an individual earns more than $50K per year. It consists of approximately 48,842 records with 14 input attributes and one target variable, offering a comprehensive set of demographic and employment-related information for model training and evaluation. Important attributes include age, education, occupation, work class, marital status, hours-per-week, capital gain, capital loss, and native country, all of which play a significant role in determining income levels.

To ensure effective model performance, the dataset is carefully pre-processed before training. Missing values are handled to maintain data consistency, categorical features are converted into numerical form using encoding techniques, and relevant attributes are selected for experimentation. The presence of both numerical and categorical data makes this dataset highly suitable for controlled experiments, enabling systematic analysis of how feature selection and parameter tuning influence the accuracy and reliability of the machine learning model.

Methodology

- Selected the Adult Income dataset for building and evaluating the machine learning model.
- Performed data preprocessing by handling missing values and encoding categorical variables.
- Chose relevant features to improve model efficiency and prediction accuracy.
- Split the dataset into training and testing sets for proper evaluation.
- Trained a supervised machine learning model using the training data.
- Conducted controlled experiments by changing one parameter at a time to analyze performance impact.
- Evaluated the model using accuracy as the primary performance metric.
- Compared results from different experiments to identify the best-performing configuration.
- Documented observations to understand how parameter tuning and feature selection optimize the model.

Train a Supervised ML Model

Training a supervised machine learning model means teaching the model using labeled data — data that already contains the correct answers (output). The model learns patterns from the input features and maps them to the target variable.

Steps:

- Collect Data: Gather a dataset with inputs and correct outputs. *Example:* Student study hours → Exam score.
- Split Data: Divide data into training set (to teach the model) and testing set (to evaluate performance). Common split: 80% training / 20% testing.
- Choose Algorithm: Select a suitable supervised algorithm such as:
  - Linear Regression (for predicting numbers)
  - Logistic Regression (for classification)
  - Decision Tree
- Train the Model: Feed the training data to the algorithm so it can learn relationships.
- Evaluate Performance: Test the model using unseen data and measure accuracy, precision, recall, etc.

Change Parameters to Observe Performance Impact

Parameters (also called hyperparameters) control how a model learns. Adjusting them can significantly improve or worsen performance.

Example:

In a Decision Tree:

- Max Depth: Controls how deep the tree grows.
  - Too deep → Overfitting (memorizes data).
  - Too shallow → Underfitting (fails to learn patterns).

Why Change Parameters?

- To improve accuracy
- To reduce errors
- To prevent overfitting and underfitting
- To make the model more generalizable
- This process is called Hyperparameter Tuning.

Common Methods:

- Grid Search
- Random Search
- Manual tuning

Compare Results with Different Feature Sets

A **feature set** is the group of input variables used to train a model.

Example:

Predicting house prices:

- Feature Set 1 → Size, Location
- Feature Set 2 → Size, Location, Number of Rooms, Age of House

After training separate models:

- Compare accuracy or error rates.
- Identify which features improve predictions.

Benefits:

- Removes unnecessary data
- Reduces training time
- Improves model accuracy
- Helps understand which factors influence predictions

Document Observations Clearly

Documentation means recording everything you discover during model training and testing.

What to Document:

- Dataset used
- Algorithm selected
- Parameter values
- Feature sets tested
- Accuracy and error results
- Problems faced
- Final conclusion

Implementation (Code & Output)

Tools Used

- Python
- Pandas
- Scikit-learn
- Documentation
- Case studies

Python Code Used

```
# Model Training with Controlled Experiments

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# STEP 1: Load Dataset

df = pd.read_csv("C:\\Users\\Anurag Rathore\\Downloads\\adult.csv")

print("Dataset Shape:", df.shape)
print(df.head())


# STEP 2: Data Preprocessing

# Handle missing values

df.replace('?', pd.NA, inplace=True)
df.dropna(inplace=True)

# Encode categorical features

le = LabelEncoder()

for col in df.select_dtypes(include='object').columns:
    df[col] = le.fit_transform(df[col])

print("\nData cleaned successfully!")
```

```
# STEP 3: Feature Sets
# Feature Set 1 (All features)

X1 = df.drop("income", axis=1)
y = df["income"]

# Feature Set 2 (Selected important features)

selected_features = ['age', 'educational-num', 'hours-per-week', 'capital-gain']
X2 = df[selected_features]



# STEP 4: Train-Test Split

X_train1, X_test1, y_train, y_test = train_test_split(
    X1, y, test_size=0.2, random_state=42)

X_train2, X_test2, _, _ = train_test_split(
    X2, y, test_size=0.2, random_state=42)



# STEP 5: Controlled Experiment 1
# Changing Hyperparameters

print("\n----- Experiment 1: Hyperparameter Change -----")

model_1 = RandomForestClassifier(n_estimators=50, random_state=42)
model_1.fit(X_train1, y_train)
pred1 = model_1.predict(X_test1)

acc1 = accuracy_score(y_test, pred1)
print("Accuracy with 50 trees:", acc1)

model_2 = RandomForestClassifier(n_estimators=150, random_state=42)
model_2.fit(X_train1, y_train)
pred2 = model_2.predict(X_test1)

acc2 = accuracy_score(y_test, pred2)
print("Accuracy with 150 trees:", acc2)
```

```
# STEP 6: Controlled Experiment 2
# Comparing Feature Sets

print("\n----- Experiment 2: Feature Set Comparison -----")

model_fs1 = RandomForestClassifier(random_state=42)
model_fs1.fit(X_train1, y_train)
pred_fs1 = model_fs1.predict(X_test1)

acc_fs1 = accuracy_score(y_test, pred_fs1)
print("Accuracy with ALL features:", acc_fs1)

model_fs2 = RandomForestClassifier(random_state=42)
model_fs2.fit(X_train2, y_train)
pred_fs2 = model_fs2.predict(X_test2)

acc_fs2 = accuracy_score(y_test, pred_fs2)
print("Accuracy with SELECTED features:", acc_fs2)



# STEP 7: Observations

print("\n----- Observations -----")

if acc2 > acc1:
   print("Increasing trees improved performance.")
else:
   print("More trees did NOT significantly improve performance.")

if acc_fs1 > acc_fs2:
   print("Using all features gives better accuracy.")
else:
   print("Selected features are sufficient and reduce complexity.")
```

Output

```
[Running] python -u "c:\Users\Anurag Rathore\OneDrive\Desktop\python projects\TASK THREE.py"
Dataset Shape: (48842, 15)
   age  workclass  fnlwgt  ... hours-per-week  native-country income
0   25    Private  226802  ...             40   United-States  <=50K
1   38    Private   89814  ...             50   United-States  <=50K
2   28  Local-gov  336951  ...             40   United-States   >50K
3   44    Private  160323  ...             40   United-States   >50K
4   18          ?  103497  ...             30   United-States  <=50K

[5 rows x 15 columns]

Data cleaned successfully!
```

```
----- Experiment 1: Hyperparameter Change -----
Accuracy with 50 trees: 0.854947484798231
Accuracy with 150 trees: 0.8574903261470426

----- Experiment 2: Feature Set Comparison -----
Accuracy with ALL features: 0.8560530679933664
Accuracy with SELECTED features: 0.8064123825317855

----- Observations -----
Increasing trees improved performance.
Using all features gives better accuracy.
```

Output Explanation

- The dataset contains 48,842 rows and 15 columns, which indicates a large dataset suitable for training a reliable supervised machine learning model.
- Data cleaning was completed successfully, meaning missing values, incorrect symbols (like ?), and inconsistencies were handled properly before training the model.
- Experiment 1 – Hyperparameter Change:
  - Accuracy with 50 trees: 85.49%
  - Accuracy with 150 trees: 85.74%
  - Increasing the number of trees slightly improved the model's predictive performance because more trees reduce variance and create stronger predictions.
- Experiment 2 – Feature Set Comparison:
  - Accuracy with all features: 85.61%
  - Accuracy with selected features: 80.64%
  - Removing some features caused a noticeable drop in accuracy, showing that the excluded features contained important information for prediction.
- The model performs well overall, achieving around 85–86% accuracy, which indicates good learning from the dataset.

Conclusion:

The supervised machine learning model was successfully trained and evaluated using different experiments. Increasing the hyperparameter (number of trees) resulted in better accuracy, proving that parameter tuning can enhance model performance. Additionally, comparing feature sets showed that using all relevant features leads to more accurate predictions than limiting the inputs.

Therefore, proper **hyperparameter tuning** and **feature selection** play a critical role in building an efficient and high-performing machine learning model. The project demonstrates that careful experimentation and observation help in optimizing model results and improving prediction reliability.

End of Report